

Antarctic Weather Platform

Technical Report

A backend and frontend system for retrieving, caching, aggregating,
and visualizing historical Antarctic meteorological observations
from AEMET OpenData

Mária Nyolcas

Prepared for GS Inima Environment

Development period: 21–28 August 2026

Abstract

This report documents the engineering reasoning behind a backend and frontend system for retrieving, caching, aggregating, and visualizing historical Antarctic meteorological observations from AEMET OpenData, built for GS Inima Environment’s evaluation of a hypothetical wind-farm project in Antarctica. It complements the repository’s `README.md`, which covers setup, usage, and a summary of design decisions, by explaining *why* each non-trivial decision was made, including the alternatives considered and their trade-offs. Source excerpts are referenced by file path and line range against the repository at the time of writing, so every claim here can be checked directly against running code, not reconstructed from memory.

A note on scope and timing

This report was written incrementally, alongside development, rather than after the fact: a deliberate choice that carried a real trade-off while the system was still in motion. Earlier revisions marked sections depending on unfinished code with a visible `TODO` rather than filling them with speculative content, and, for a period, stated plainly that no CI pipeline existed rather than describing one that had not yet been built. By this revision, every subsystem described below (the AEMET integration, timezone and DST handling, aggregation, SQLite caching, the frontend, the test suite, and continuous integration itself, Section 10) is complete, and every section is written against the actual, running configuration, not intent.

Contents

1	Executive Summary	4
2	Requirements Analysis	4
2.1	Source of truth	4
2.2	Core functional requirements, as specified	4
2.3	Ambiguities identified before implementation	5
2.4	Assumptions requiring verification	5
3	Architecture	5
3.1	Layering, and the requirement it serves	5
3.2	Orchestration and dependency injection	6
3.3	Three model shapes, not one	6
4	AEMET Integration	7
4.1	What had to be verified before writing the client	7
4.2	Undocumented behavior discovered only by live testing	7
4.3	Two distinct meanings of <code>estado: 404</code> , and why conflating them would be a silent bug	8
4.4	Retry and rate-limiting strategy	9
4.5	Generic retry for transport and server failures	9
5	Timezone and DST Handling	10
5.1	Four concepts, not one	10
5.2	Resolving a timezone input	10
5.3	The two DST hazards, handled explicitly rather than left to Python's defaults . .	11
5.4	Output representation	11
6	Aggregation Strategy	12
6.1	Bucket boundaries are drawn in Madrid local time, not UTC	12
6.2	Why the mean, and why wind speed additionally reports a maximum	12
6.3	Quality filtering happens once, at aggregation time	13
6.4	Sparse buckets, not padded ones	13
7	Persistence and Caching	13
7.1	Two tables, each tracking a different fact	13
7.2	Cache-hit logic: single-range containment, chosen deliberately over a general interval algorithm	14
7.3	Chunking interacts with caching, not just with AEMET's range limit	14
7.4	Two SQLite-specific correctness bugs, caught by tests rather than assumed away .	15
7.5	Cache staleness: intentionally unbounded	15
7.6	The <code>latest-available</code> endpoint	15
8	Testing Strategy	16
8.1	Backend: 142 tests, isolating the two riskiest subsystems	16
8.2	Frontend: 98 tests, behavioral rather than implementation-focused	17
9	Frontend Architecture	17
9.1	State as one discriminated union, not several booleans	17
9.2	Component decomposition follows analytical weight, not file-size convenience . .	17
9.3	The hero and the transition to the analytical workspace	18

10 Continuous Integration and Engineering Quality	19
11 Security Considerations	22
12 Scalability Considerations	22
13 Trade-offs and Limitations	23
14 Future Improvements	23
14.1 Hardening: closing the gaps already stated in this report	24
14.2 Further ideas: growing beyond the challenge’s current scope	24
14.2.1 A dedicated mobile experience, not just a responsive layout	24
14.2.2 Database management: from a single SQLite file to an operable store . . .	24
14.2.3 Timezone and location adaptability beyond the two Antarctic stations . .	25
15 Conclusion	25

List of Figures

1	The idle-state landing page: the full-bleed polar hero with the station-marked coastline illustration, the query form, and the three measurement explainer cards, before any query has been submitted. The timezone field defaults to Europe/Madrid , and both date fields already show a “data available through” caption sourced from <code>GET /observations/latest-available</code> (Section 7.6).	18
2	A successful query for Gabriel de Castilla, daily aggregation, January 2026: the compact header (replacing the hero, never leaving the page headless), the query form retaining the submitted values, the KPI strip with color-coded accent rails per measurement family, and the three synchronized time-series panels (temperature, wind speed mean/max, pressure). The dashed line in the wind panel is the bucket maximum, sharing wind’s own hue rather than a separate color, so “mean vs. max” reads as one measurement family, not two (Section 6).	20
3	The same query, scrolled further down: the wind-energy analysis section (mean/-max summary, a reused wind time series, and a client-side histogram of the wind-speed distribution), the query-metadata summary strip, and the observations table. No turbine-specific threshold (cut-in, rated, or cut-out speed) is printed anywhere in this view, deliberately (Section 13).	21

1 Executive Summary

The system exposes a single primary endpoint, `GET /observations`, that retrieves historical weather observations for one of two Spanish Antarctic research stations (Gabriel de Castilla, Juan Carlos I) over a caller-specified date range, optionally aggregated to hourly, daily, or monthly resolution, and returns temperature, pressure, and wind speed in a consistent, timezone-normalized shape.

Three engineering concerns dominated the design, in the order the challenge itself weights them:

1. **Correctness under real-world data conditions.** AEMET’s actual API behavior, including a two-step response indirection, a "NaN" string sentinel for missing measurements, an undocumented data-quality flag, and station-dependent optional fields, was verified against the live API before any client code was written, rather than assumed from the endpoint’s description alone (Section 4).
2. **Temporal correctness.** The system distinguishes an instant, a local wall-clock representation, a named timezone’s DST rules, and a fixed UTC offset as four genuinely different concepts, and handles the two DST edge cases (nonexistent and ambiguous local times) explicitly rather than relying on Python’s implicit defaults (Section 5).
3. **A caching strategy whose behavior is fully explainable.** Rather than a general interval-coverage algorithm, the cache uses a deliberately scoped single-range containment check, reasoned through as a trade-off between completeness and explainability (Section 7).

2 Requirements Analysis

2.1 Source of truth

The authoritative requirements are the official challenge specification (PDF, three parts) together with direct clarification from the hiring contact. Where the two could be read differently, the original PDF is treated as authoritative, since it is the actual assessment document rather than a paraphrase of it.

2.2 Core functional requirements, as specified

- A Python web API (FastAPI or Flask) exposing the AEMET Antarctic endpoint’s data.
- Caller-specified start and end datetime (`AAAA-MM-DDTHH:MM:SS`).
- An optional location/timezone parameter, expressible either as an IANA timezone name (e.g. `Europe/Berlin`) or as a fixed UTC offset (e.g. `+02:00`).
- A choice of exactly two supported stations: “Gabriel de Castilla” and “Juan Carlos I”.
- A time-aggregation selector: None, Hourly, Daily, or Monthly, with daily/monthly aggregation required to respect the meteo station’s timezone rather than grouping raw UTC timestamps.
- Zero to three requested measurement types (temperature, pressure, wind speed); zero selected means all three are returned.
- Output datetimes in `Europe/Madrid` (CET/CEST), always carrying an explicit UTC offset.
- Explicit confirmation of DST behavior.
- SQLite persistence to avoid redundant upstream requests, with attention to database best practice, motivated explicitly by an anticipated high-volume, latency-sensitive consumer (intraday traders).
- Structured logging for troubleshooting.
- Automated tests, not required to be exhaustive.
- A frontend allowing the data to be queried and viewed in a table, optionally a chart, with explicit permission to use any reasonable technology stack.

2.3 Ambiguities identified before implementation

The specification is precise about behavior but silent on several implementation-level questions that materially affect correctness. Each was resolved deliberately, documented as an assumption, and isolated behind a narrow interface so it can be revisited without a wider rewrite.

Aggregation statistic. The specification requires aggregation but does not name a statistic. Arithmetic mean was chosen for all three measurements (Section 6); a second question — whether wind speed specifically should also report a maximum, given the wind-farm context — was raised with the assigning team rather than decided unilaterally, since it changes the response contract.

Default input timezone. The specification marks the timezone parameter optional without stating a default. **Europe/Madrid** was chosen over UTC specifically because it keeps an unspecified date range aligned with the same calendar day in the (unconditionally Madrid) output, rather than spilling across a day boundary — a concrete failure mode that was worked through with a numeric example before the decision was made.

Data-quality flag. AEMET’s live response includes a `qdato` field (data quality, 0 = good, 1 = bad) that is not part of the specification’s field table. Its existence was discovered only by making a live API call, not by reading documentation. Whether a flagged-bad reading should be silently excluded from aggregation or exposed unfiltered was likewise raised with the assigning team.

Cache staleness. The specification states AEMET’s Antarctic dataset updates only a few times a year, but does not say whether a previously cached range should ever be considered stale and re-fetched. The current implementation caches a fetched range indefinitely; whether a revalidation window is expected was raised with the assigning team rather than assumed.

2.4 Assumptions requiring verification

Two further points were resolved by direct experimentation against the live AEMET API rather than left as assumptions, and are recorded here because they materially shaped the client and domain model:

- Whether the AEMET Antarctic endpoint returns data directly or via an indirection (a documented risk called out by the assigning team) — confirmed to be a two-step indirection (Section 4).
- The exact station identifiers for the two required stations, and whether the field schema is uniform across them — confirmed via AEMET’s own OpenAPI specification and a live request per station; the two stations were found *not* to return an identical field set (Section 4).

3 Architecture

3.1 Layering, and the requirement it serves

The backend is organized into five layers, each with one responsibility:

Layer	Responsibility
API / Routes	Request validation, HTTP concerns, response shaping
Application / Service	Orchestration: cache check → AEMET fetch → aggregate
Domain	Pure functions: timezone conversion, aggregation, quality filtering
Repositories / Integrations	AEMET client, SQLite repository
External systems	SQLite, AEMET OpenData

This is not layering for its own sake. The concrete requirement it serves is testability under the specific correctness risks this system has: the domain layer (`app/domain/time.py`, `app/domain/aggregation.py`) has *no* dependency on FastAPI, SQLAlchemy, or httpx, so the timezone and DST logic — the single riskiest part of this system, per Section 5 — can be unit tested by constructing plain Python `datetime` values and asserting on plain return values, without a database, an HTTP mock, or a running application. The same is true of the aggregation module. A change to how AEMET responses are parsed, or how the API formats a response, cannot silently change how a DST transition is handled, because the code paths do not intersect.

3.2 Orchestration and dependency injection

The application/service layer is a single class, `WeatherService` (`backend/app/services/weather_service.py`), whose entire responsibility is sequencing: check whether the requested range is cached, call AEMET only on a miss, persist the result, then aggregate and return. It depends on its collaborators through its constructor rather than constructing them itself:

```

1 class AemetObservationSource(Protocol):
2     """The subset of AemetClient this service needs, lets tests substitute a
3         fake with no real HTTP client behind it."""
4
5     async def get_observations(
6         self, station: Station, start: datetime, end: datetime
7     ) -> list[StationObservation]: ...
8
9 class WeatherService:
10     """Orchestrates cache lookup, AEMET retrieval, and aggregation."""
11
12     # Exceptions from either collaborator propagate unchanged; mapping
13     # them to HTTP responses is the API layer's job, not this one's.
14     def __init__(
15         self, aemet_client: AemetObservationSource, repository:
16             ObservationRepository
17     ) -> None:
18         self._aemet_client = aemet_client
19         self._repository = repository

```

Listing 1: `WeatherService` depends on an AEMET source through a `Protocol`, not the concrete client class. `backend/app/services/weather_service.py`, lines 12–29.

This is a direct application of dependency inversion, chosen deliberately over the alternative of having `WeatherService` import and instantiate `AemetClient` directly. The concrete alternative that was rejected — constructing the real client inside the service and mocking `httpx` at the transport level in every service test — would work, but it means every service-layer test also exercises (and can be broken by) HTTP mocking concerns that have nothing to do with the orchestration logic actually being tested. Depending on a `Protocol` means the test suite’s fake client (`FakeAemetClient`, `tests/unit/test_weather_service.py`, lines 16–27) is a small class with no knowledge of HTTP at all, and — because `Protocol` is structurally typed — `mypy` verifies that both the fake and the real `AemetClient` satisfy the same interface, so this is a property enforced by the type checker, not merely true by convention.

3.3 Three model shapes, not one

A single AEMET observation is represented by three distinct types as it moves through the system, and this distinction is deliberate rather than incidental:

1. **Transport DTO** (`AemetObservationDTO`, `app/integrations/aemet/schemas.py`) — mirrors AEMET’s actual wire format, including its quirks (Section 4).

2. **Domain model** (`StationObservation`, same file) — a frozen, verified, in-domain reading with no knowledge of AEMET’s field names or serialization conventions.
3. **API response model** (`ObservationResponse`, `app/api/schemas.py`) — the wire contract this application exposes, independent of both AEMET’s format and the internal domain representation.

Collapsing these into one type would couple three independently-changing concerns: AEMET could alter its response format, the internal representation could be refactored for a new aggregation strategy, and the public API contract could evolve for frontend needs — none of which should require touching the other two.

4 AEMET Integration

4.1 What had to be verified before writing the client

The challenge specification flagged the AEMET Antarctic endpoint as carrying real integration risk, without saying exactly what that risk was. Three concrete unknowns were resolved against the live API, using a personal AEMET OpenData API key, before any client code was written, on the principle that a typed client built against an assumed shape would need to be rewritten anyway the moment reality disagreed:

1. **Response indirection.** AEMET’s Antarctic endpoint (`GET /api/antartida/datos/fechaini/{...}/f/estacion/{...}`) does not return observation data directly. It returns a small JSON envelope containing two pre-signed URLs, `datos` (the actual records) and `metadatos` (field documentation), which must be fetched with a second HTTP call. This two-step shape is common across AEMET OpenData endpoints generally but is not obvious from the Antarctic endpoint’s own documentation page, and was confirmed by making a real request and inspecting the response body directly.
2. **The `datos` URL does not take the API key.** The pre-signed link AEMET issues in the envelope is itself the authentication mechanism for the second request; attaching the `api_key` header (correct for the first call) to the second call was tried and made no difference, confirming the header is simply ignored on that endpoint rather than rejected, which is why `_request_records` (`client.py`, lines 113–128) passes `headers=None` explicitly rather than omitting the parameter by accident.
3. **Station identifiers and field uniformity.** The two required station names, “Gabriel de Castilla” and “Juan Carlos I”, are not self-evidently mapped to AEMET’s internal station codes. Cross-referencing AEMET’s published OpenAPI specification against a live request per station confirmed 89070 and 89064 respectively, and, more importantly, revealed the two stations do *not* return an identical field set: Juan Carlos I’s live response included `radKjM2`, `rec`, `tsb`, and `altNieve` fields entirely absent from Gabriel de Castilla’s. Had the transport DTO been written to expect a fixed field set, this would have surfaced as a runtime validation failure against exactly one of the two required stations, in production, rather than being caught at the boundary. The mapping layer instead treats every field outside the five actually used (station identifier, timestamp, temperature, pressure, wind speed, plus the quality flag below) as optional and ignored.

4.2 Undocumented behavior discovered only by live testing

Two further AEMET behaviors were never documented anywhere in the specification or the OpenAPI file, and would have produced either silently wrong data or a confusing failure if the client had been written against documentation alone.

The quality flag. Every observation record carries a `qdato` field (0 = good, 1 = bad), not

part of the challenge’s stated field table. Its existence was discovered only by inspecting a live response body. Whether a flagged-bad reading should be silently dropped or surfaced as-is materially changes the API’s output, so this was raised with the assigning team rather than decided unilaterally (Section 2): the resolution is that a `qdato = 1` reading is retained verbatim in the SQLite cache (the cache is a faithful copy of what AEMET actually sent) but reported as `null` for that measurement in every API response, aggregated or not, since a flagged-bad value should never be presented to a caller as trustworthy.

The "NaN" sentinel. AEMET represents an inapplicable or missing numeric field as the literal JSON string "NaN", not a JSON `null` and not an omitted key. A transport model expecting a numeric type or `null` would raise a validation error on any record containing this sentinel, which is common enough (not every station reports every field on every reading) that it would have broken ordinary queries, not just edge cases. This is intercepted explicitly in the DTO-to-domain mapping, before any numeric coercion is attempted, converting the literal string to `None` rather than letting it reach a `float()` call or a Pydantic numeric validator.

4.3 Two distinct meanings of `estado: 404`, and why conflating them would be a silent bug

AEMET signals “no data available for this range” in two different ways, both found empirically rather than documented:

1. A genuine HTTP 404 status code, handled by the caller before the envelope is even parsed (`_request_envelope`, lines 96–111).
2. An HTTP 200 response whose JSON body carries `estado: 404` with no `datos/metadatos` fields at all.

The second form is also, confusingly, the exact same wrapper AEMET uses when a request is rejected for spanning more than its undocumented ~31-day maximum range (“`El rango de fechas no puede ser superior a 1 mes`”). This was discovered by querying a full year of data and receiving far fewer observations than expected, silently, with no error raised, because the naive interpretation of `estado: 404` as “empty result” swallowed the actual problem (a rejected, oversized request) and reported it as if AEMET simply had no data for most of the year. Treating these as the same case would be a genuine correctness bug, not a coarse approximation: it would make a query silently return a small, wrong subset of a valid range while reporting success. `_parse_envelope` (`client.py`, lines 206–232) distinguishes the two by inspecting the `descripcion` text for the range-length phrase before falling back to treating the response as a legitimate empty result, and raises `AemetRangeTooLongError` rather than returning `[]` when it matches:

```
1 if isinstance(body, dict) and body.get("estado") == 404:
2     descripcion = body.get("descripcion", "")
3     if isinstance(descripcion, str) and "rango de fechas" in descripcion.lower():
4         raise AemetRangeTooLongError(descripcion)
5     return None
```

Listing 2: Disambiguating two meanings of the same wrapper. `backend/app/integrations/aemet/client.py`, lines 221–225.

Because `WeatherService` proactively splits every requested range into ≤ 31 -day chunks before calling the client at all (Section 7), `AemetRangeTooLongError` should never actually surface during normal operation; it exists so that if that invariant is ever violated by a future change, the client fails loudly with a distinct, named exception rather than silently misreporting a rejected request as an empty one.

4.4 Retry and rate-limiting strategy

AEMET does not publish a rate limit for this endpoint. Per explicit guidance from the assigning team, the chosen response to an undocumented limit is to behave conservatively rather than to guess a threshold and tune against it empirically. The client applies two independent mechanisms:

Proactive throttling. Every outbound request, regardless of outcome, passes through a single choke point (`_throttled_get`, lines 177–185) enforcing a minimum 500ms spacing from the previous request. This is guarded by an `asyncio.Lock`, not merely a plain variable check, because an unsynchronized “has enough time passed?” check followed by a sleep is a classic check-then-act race: two concurrent callers could both observe that enough time has elapsed and fire simultaneously, defeating the throttle exactly when it matters most (concurrent load).

Reactive backoff on a real 429. A single retry after a fixed delay is sufficient for the generic 5xx/timeout case (Section 4.5 below), but a 429 specifically needed more. `WeatherService` splits any range over ~31 days into sequential chunked requests (Section 7), so a single wide query can issue dozens of requests to AEMET in a row; this was confirmed live to be able to trip AEMET’s undocumented limit more than once within a single logical query, not just once at the start. A single retry, tried first, was empirically insufficient: verified directly by running the exact 26-month query that had been failing and observing that a real 429 could still occur again after the first retry succeeded. The fix retries up to three times, with the cooldown doubling on each consecutive attempt (5s, 10s, 20s by default):

```
1 if response.status_code == 429:
2     for attempt in range(1, _MAX_RATE_LIMIT_RETRIES + 1):
3         cooldown = self._rate_limit_cooldown_seconds * (2 ** (attempt - 1))
4         self._apply_rate_limit_cooldown(cooldown)
5         logger.warning(
6             "AEMET returned 429, retry %d/%d after %.1fs cooldown: %s",
7             attempt, _MAX_RATE_LIMIT_RETRIES, cooldown, url,
8         )
9         await asyncio.sleep(cooldown)
10        response = await self._throttled_get(url, headers=headers)
11        if response.status_code != 429:
12            break
```

Listing 3: Multi-retry 429 handling with a doubling cooldown. `backend/app/integrations/aemet/client.py`, lines 146–167.

This was verified end-to-end against the live API, not only against mocked 429 responses in the unit test suite: the same 26-month query that had previously failed outright was re-run against production AEMET after the fix, encountered one real 429 partway through its sequence of chunked requests, retried successfully on the first attempt, and completed with 109,832 observations returned, confirmed against the backend’s own log output rather than assumed from a lack of visible error. A 429 that persists through all three retries still raises `AemetUnavailableError` rather than retrying indefinitely, since an AEMET-side block that outlasts a 35-second total backoff window is a genuine outage, not a transient collision.

4.5 Generic retry for transport and server failures

Connection failures, timeouts, and 5xx responses are retried exactly once, after a short fixed delay, deliberately not the same doubling-backoff strategy used for 429s. The reasoning is proportionality to what this application actually is: a single-user local service, not a distributed system fielding concurrent load from many clients, so a transient network blip is the failure mode being guarded against, not a shared upstream buckling under aggregate pressure that would call for exponential backoff. A second attempt either succeeds quickly or the caller learns AEMET is genuinely unavailable and receives `AemetUnavailableError` rather than waiting through several

escalating delays for a condition unlikely to self-resolve on this timescale. 401/403 responses (a rejected or missing API key) are never retried and raise `AemetAuthenticationError` instead, kept as a distinct exception from `AemetUnavailableError`: a bad key is a configuration problem that retrying cannot fix, and conflating the two would give any future retry-on-failure logic upstream the wrong answer to “is this worth trying again?”

5 Timezone and DST Handling

5.1 Four concepts, not one

The specification requires the system to accept an optional timezone (as either an IANA name or a fixed UTC offset), always output `Europe/Madrid` regardless of the input, and explicitly confirm DST behavior. Meeting this correctly requires keeping four genuinely different concepts distinct throughout the pipeline, rather than treating “timezone” as one idea with several representations:

1. A **naive wall-clock value**: what a caller types (2026-03-29T02:30:00), meaningless as an instant in time until paired with a timezone.
2. A **named timezone** (`zoneinfo.ZoneInfo`): an IANA DST rule set that produces a different UTC offset on different calendar dates, not a fixed number.
3. A **fixed UTC offset** (e.g. +02:00): exactly that many hours from UTC on every date supplied, carrying no DST logic at all. This is not a simplified way of writing “Europe/-Madrid in summer”; it is a different kind of object entirely, and a caller using this form is opting out of DST reasoning explicitly, a consequence stated directly in the code comment above `_UTC_OFFSET_PATTERN` (`time.py`, lines 14–19).
4. The **UTC instant** that a (wall-clock, timezone) pair resolves to, and the **Europe/Madrid output representation** of that same instant, which is unconditional: every response datetime is Europe/Madrid regardless of what timezone the request specified.

Collapsing any two of these would reintroduce exactly the class of bug this system is trying to rule out: for instance, treating a fixed offset as interchangeable with an IANA name would make a caller’s +01:00 input silently behave as if it carried Madrid’s DST rules on a date where Madrid is actually at +02:00 (CEST), producing a response an hour off from what was actually requested with no error raised anywhere.

5.2 Resolving a timezone input

`resolve_timezone` (`time.py`, lines 23–33) tries the fixed UTC offset pattern first, falling back to `ZoneInfo` construction, and raises `InvalidTimezoneError` (mapped to HTTP 400) for anything that matches neither form, rather than letting an unrecognized string propagate as an unhandled `ZoneInfoNotFoundError` deep inside the conversion pipeline:

```
1 def resolve_timezone(name: str) -> tzinfo:
2     offset_match = _UTC_OFFSET_PATTERN.match(name)
3     if offset_match:
4         sign, hours, minutes = offset_match.groups()
5         delta = timedelta(hours=int(hours), minutes=int(minutes))
6         return timezone(-delta if sign == "-" else delta)
7
8     try:
9         return ZoneInfo(name)
10    except ZoneInfoNotFoundError as exc:
11        raise InvalidTimezoneError(name) from exc
```

Listing 4: Two accepted timezone forms, tried in order. `backend/app/domain/time.py`, lines 23–33.

5.3 The two DST hazards, handled explicitly rather than left to Python’s defaults

Python’s `datetime` module does not, by default, protect a caller from either DST edge case; both are handled explicitly in `to_utc_instant` (`time.py`, lines 36–55) rather than relying on whatever `zoneinfo` happens to pick.

Nonexistent local times. During a spring-forward transition (in 2026, confirmed empirically against Python’s own `zoneinfo` data rather than assumed, Europe/Madrid’s clocks jump from 02:00 to 03:00 on 29 March), a wall-clock time inside the skipped hour, e.g. 2026-03-29T02:30:00, never actually occurs. Python does not reject this at construction time; it silently assigns it some UTC offset and lets the program proceed as if the time were valid. The function detects this by round-tripping the candidate value through UTC and back: if converting to UTC and back to the same named timezone does not reproduce the original wall-clock value bit-for-bit, the value was never real, and `NonexistentLocalTimeError` is raised (mapped to HTTP 400) rather than silently returning whichever instant `zoneinfo`’s default `fold=0` behavior happened to produce:

```
1 aware = local_dt.replace(tzinfo=tz, fold=0)
2
3 round_tripped = aware.astimezone(ZoneInfo("UTC")).astimezone(tz)
4 if round_tripped.replace(tzinfo=None) != local_dt:
5     raise NonexistentLocalTimeError(local_dt.isoformat(), str(tz))
6
7 return aware.astimezone(ZoneInfo("UTC"))
```

Listing 5: Detecting a spring-forward gap by round-tripping. `backend/app/domain/time.py`, lines 45–55.

Ambiguous local times. During the fall-back transition (Madrid’s clocks move from 03:00 back to 02:00 on 25 October 2026), the wall-clock hour between 02:00 and 03:00 occurs twice: once at UTC+2 (CEST, before the transition) and once at UTC+1 (CET, after it). Python’s `fold` attribute distinguishes the two occurrences, but leaving the default (`fold=0`) unaddressed would mean the choice is made by a Python implementation detail rather than a documented decision. This system resolves every ambiguous time to its *first* occurrence (pre-transition, `fold=0`) explicitly, as a stated convention, not an implicit default the caller has to discover by reading CPython source.

Both transition dates for 2026 were verified against Python’s installed `zoneinfo` (IANA tz database) directly, not assumed from a general rule about “last Sunday of March/October”, and both hazards are covered by tests anchored to those exact calendar dates (`backend/tests/unit/test_domain_time.py`), so a future tz database update that shifted a transition date would be caught by a failing test rather than silently changing behavior.

5.4 Output representation

`to_output_representation` (`time.py`, lines 58–66) converts any aware instant to Europe/-Madrid, unconditionally, and rejects a naive input outright rather than guessing which timezone it might be in. This is the only function in the pipeline responsible for producing the representation callers actually see, so every response datetime, and every aggregation bucket boundary, is computed by calling through this one function rather than by ad hoc `.astimezone()` calls scattered across the codebase.

6 Aggregation Strategy

6.1 Bucket boundaries are drawn in Madrid local time, not UTC

The specification requires that daily and monthly aggregation respect the station’s (i.e. Madrid’s, per the fixed output timezone) local calendar, not raw UTC grouping. This matters concretely, not just as a stated rule: because Madrid’s UTC offset is never zero (+01:00 CET or +02:00 CEST), a boundary drawn against raw UTC timestamps would routinely split observations that belong to the same Madrid calendar day into two different buckets, or merge observations from two different Madrid days into one, depending on the time of year and time of day. `aggregate` (`aggregation.py`, lines 78–131) converts every observation to its Madrid representation *before* computing a bucket key (`_bucket_key`, lines 46–53), so the grouping key is always a local calendar tuple (year, month, day, hour for hourly; year, month, day for daily; year, month for monthly), never a UTC timestamp:

```
1 for obs in observations:
2     local_dt = to_output_representation(obs.observed_at)
3     key = _bucket_key(local_dt, level)
4     buckets[key].append(obs)
5     bucket_starts.setdefault(key, _bucket_start(key, local_dt))
```

Listing 6: Bucket keys are Madrid calendar tuples. `backend/app/domain/aggregation.py`, lines 100–104.

This is tested directly rather than trusted by inspection: two observations exactly one hour apart in UTC, straddling a Madrid local midnight, are asserted to land in different daily buckets, and the same scenario is repeated across the March 2026 DST transition date to confirm the bucket key tracks the local calendar date through the offset change, not a fixed-width UTC split (`test_domain_aggregation.py`).

6.2 Why the mean, and why wind speed additionally reports a maximum

The specification requires aggregation but names no specific statistic. Arithmetic mean,

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i,$$

was chosen for all three measurements as the standard estimator of a continuous physical quantity’s typical value over an interval, appropriate given AEMET’s roughly regular ~10-minute sampling interval. It is, however, an explicitly incomplete summary on its own: averaging discards information about variability within the bucket, most consequentially for wind speed, where a bucket containing one strong gust amid mostly calm air can share an identical mean with a bucket of steady, moderate wind.

This is not a purely statistical concern in this context. It was raised with, and confirmed by, the assigning team: turbine power generation has minimum (cut-in), maximum (cut-out), and optimal (rated) operating wind-speed thresholds, so a mean alone can conceal a period whose actual wind conditions were outside a turbine’s safe or productive operating range, which is directly load-bearing for a wind-farm feasibility assessment in a way it is not for, say, an average temperature. The resolution changes the response contract itself, not merely an internal calculation: `AggregatedObservation` (lines 19–38) carries both `wind_speed_ms` (the bucket mean) and `wind_speed_max_ms` (the bucket maximum) as separate fields, while temperature and pressure report only their mean, since the same operational-threshold reasoning does not apply to them in this context.

6.3 Quality filtering happens once, at aggregation time

AEMET’s `qdata` flag is applied at exactly one point in the pipeline (`aggregate`, not the repository layer): every mean and maximum is computed only over readings where `is_good_quality` is true and the value itself is not `None`. A bucket where every contributing reading is missing or flagged bad reports `null` for that measurement, deliberately never `0.0`: zero is a real, physically meaningful value a sensor could legitimately report, so using it to mean “no valid data” would make a genuine zero-reading bucket indistinguishable from an empty one, silently corrupting downstream analysis (a chart, a mean-of-means) with no signal that anything was wrong. Concentrating this filtering in one function, rather than applying it once when data is cached and again when it is read back, means the policy can be revisited (e.g. if the assigning team’s stance on flagged-bad data ever changed) without needing to re-fetch or migrate anything already stored; `ObservationRepository._to_domain` (`repository.py`, lines 105–117) deliberately returns raw stored values, unfiltered, so this single point of enforcement is not duplicated or risked drifting out of sync.

6.4 Sparse buckets, not padded ones

`aggregate` only emits a bucket for a calendar period that contains at least one raw observation; a gap in AEMET’s own data (a missing hour, a station outage) produces no row for that period at all, rather than a row padded with `nulls` to preserve a continuous time axis. This keeps the function’s contract simple, since it needs only the observations themselves and never the originally requested range, and avoids fabricating rows the source data never actually contained. The trade-off is explicit: a chart or table consuming this response must not assume a gap-free, evenly-spaced time axis, and the frontend’s chart component (Section 9) is written with that assumption in mind, most visibly in not connecting a line across a `null` value.

7 Persistence and Caching

7.1 Two tables, each tracking a different fact

The schema (`backend/app/db/models.py`) has exactly two tables, and each exists to answer a specific question the caching logic actually needs answered, rather than being a generic denormalized store:

`observations` answers “what did AEMET report for station *X* at instant *t*?” Its identity is the pair (`station`, `observed_at`): the same station measuring at the same instant is the same observation by definition, which is enforced as an actual database constraint,

```
1 __table_args__ = (  
2     UniqueConstraint("station", "observed_at", name="uq_observation_identity"),  
3     Index("ix_observation_station_time", "station", "observed_at"),  
4 )
```

Listing 7: The observation’s real identity, enforced by the database, not just application convention. `backend/app/db/models.py`, lines 41–44.

not merely an application-level convention that a caching bug could quietly violate. The same composite column pair also serves as the query index, since every real read is “observations for station *X* between times *A* and *B*”, so the constraint and the index serve two purposes with one structure.

`fetch_ranges` answers a different question: “was AEMET ever asked about station *X* for this range, regardless of what it returned?” This table is necessary specifically because AEMET publishes this dataset with a real, variable lag behind the present. A recent date range can

legitimately have zero observations simply because AEMET has not published that data yet, and without an explicit record that the range was already queried, that state would be indistinguishable from “this range was never asked about”, which would cause every request for a recent range to re-query AEMET on every call, defeating the cache’s purpose for exactly the queries most likely to be repeated (a user checking whether new data has appeared).

7.2 Cache-hit logic: single-range containment, chosen deliberately over a general interval algorithm

For a requested range R on a given station, `is_range_cached` (`repository.py`, lines 18–33) checks whether R is fully contained within a *single* previously fetched range, not whether R is covered by the union of several stored ranges (a true interval-tree set difference, $R \setminus \bigcup C_i$, across arbitrarily many stored intervals). A request spanning two previously-fetched but non-adjacent ranges is therefore treated as a full miss and re-fetched in its entirety, even though part of that range is technically already present in `observations`.

This is a deliberate, scoped trade-off, not an oversight. The actual usage pattern this system serves, a user submitting and then refining one query at a time, is well served by single-range containment: it is simple to state, simple to verify by reading the query, and does not require maintaining or periodically compacting a set of disjoint stored intervals. A general interval-merging cache would be strictly more capable in the abstract, but that capability is not free: it introduces a real algorithm (interval merging or an interval tree) with its own edge cases (adjacent but non-overlapping ranges, redundant sub-ranges, merge ordering) purely to serve a usage pattern this application does not actually have. Reasoning explicitly about proportionality here is the same judgment applied to the layering decision in Section 3 and the DDD scoping decision in the README: correctness and explainability do not require the more general mechanism at this scale, so it is not built.

7.3 Chunking interacts with caching, not just with AEMET’s range limit

Because AEMET rejects any single request spanning more than ~ 31 days (Section 4), `WeatherService.get_obs` splits every requested range into sequential ≤ 31 -day chunks *before* either the cache or AEMET is consulted, and checks the cache independently for each chunk:

```

1 for chunk_start, chunk_end in _chunk_range(start, end):
2     if self._repository.is_range_cached(station, chunk_start, chunk_end):
3         logger.info("Cache hit for %s [%s, %s]", station.value, chunk_start,
4                     chunk_end)
5         continue
6     fetched = await self._aemet_client.get_observations(station, chunk_start,
7                                                         chunk_end)
8     self._repository.store_fetch_result(station, chunk_start, chunk_end, fetched)

```

Listing 8: Cache coverage is checked per chunk, giving finer-grained reuse than the containment rule alone would suggest. `backend/app/services/weather_service.py`, lines 82–94.

This has a consequence worth stating explicitly: although the containment rule above is scoped to a single stored range, chunking makes the *effective* cache behavior finer-grained than that description alone implies. A year-long request previously fetched one month at a time is checked, and can hit, one month-sized chunk at a time; only the chunks not already covered trigger a real AEMET call. A test asserts this directly, storing one cached month inside a larger requested span and confirming only the uncached remainder results in an AEMET call (`test_already_cached_chunk_within_a_large`, `test_weather_service.py`).

7.4 Two SQLite-specific correctness bugs, caught by tests rather than assumed away

Two issues surfaced during development that would have been silent data bugs, not crashes, had they gone unnoticed, and are recorded here because both were caught by a failing test rather than anticipated in advance.

Timezone loss on round-trip. SQLite’s `DATETIME` column type has no concept of a timezone; SQLAlchemy, by default, returns a naive `datetime` on read even when an aware value was written. A round-trip test (write an aware UTC datetime, read it back, assert `tzinfo` is still set) caught this directly: the value came back missing its timezone entirely. Given the amount of this system built specifically to keep timezone-aware and naive values from ever being conflated (Section 5), silently reintroducing a naive value at the persistence boundary would have undermined that work in a way that would not necessarily surface as a visible error, only as an occasionally wrong answer. The fix, `UTCDateTime` (`models.py`, lines 17–34), is a small SQLAlchemy `TypeDecorator` that strips the timezone before writing (SQLite cannot store it) and re-attaches UTC on every read, since every datetime this application ever persists is UTC by construction; this makes the fix a correctness guarantee actually enforced by a type, not a convention relying on every call site remembering to reattach it manually.

Session.merge() upserts on the wrong key. An idempotency test, re-fetching a range that overlapped a previously-stored one, was expected to update the existing rows; instead it raised `IntegrityError` against the `(station, observed_at)` unique constraint. The cause: SQLAlchemy’s `Session.merge()` decides insert-versus-update based on the ORM’s primary key (`id`), and every freshly constructed `ObservationRecord` coming back from a new AEMET fetch has no `id` set yet, so `merge()` always treated it as a new row regardless of whether the same `(station, observed_at)` pair already existed. The fix, in `store_fetch_result` (`repository.py`, lines 69–102), replaces `merge()` with an explicit `INSERT ... ON CONFLICT (station, observed_at) DO UPDATE`, which targets the pair that is actually the observation’s real business identity rather than the ORM surrogate key, and both writes (the observation rows and the `FetchRange` marker) are wrapped in a single `session_scope` transaction so the two tables can never disagree about what has been cached if one write succeeds and the other fails.

7.5 Cache staleness: intentionally unbounded

A fetched range is cached indefinitely; there is no TTL or revalidation policy on the `observations` table. This was confirmed as correct by the assigning team rather than assumed for convenience (Section 2): AEMET’s historical Antarctic dataset is stable once published and does not change retroactively, so a revalidation policy would add real complexity (a staleness clock, a re-fetch trigger, cache-invalidation reasoning) to guard against a condition that does not occur in this data source. This is a different concern from the `get_latest_available_date` endpoint’s own 1-hour in-process TTL (Section 7.6), which caches the answer to a different, genuinely time-varying question, “what is the most recent date with data *right now*”, not the historical observations themselves.

7.6 The latest-available endpoint

A later addition to the system, `GET /observations/latest-available` answers “what is the most recent date AEMET is confirmed to have data for?”, driving the frontend’s date-picker cap so a query doomed to return nothing is caught before submission rather than after. AEMET itself exposes no such endpoint; it is a pure range-query API. The implementation is cache-first: `MAX(observed_at)` over the already-cached `observations` table is both free (no network call) and already the most authoritative record this application has of what data has actually been confirmed to exist, since the cache is populated exclusively by successful AEMET fetches.

Only when nothing is cached yet for a station (a cold start) does the service fall back to a live probe, stepping back through 60-day windows for up to six attempts looking for the first window that returns any data (`_probe_aemet_for_latest_date`, `weather_service.py`, lines 125–138), bounded so a station with no AEMET coverage at all fails after a fixed number of attempts rather than probing indefinitely. The endpoint’s own answer is additionally cached in-process for one hour per station, since the true value changes at most roughly monthly in reality but this endpoint may be called on every frontend page load.

8 Testing Strategy

8.1 Backend: 142 tests, isolating the two riskiest subsystems

The backend test suite (`backend/tests/`, `pytest`, `pytest-asyncio`, `pytest_httpx` for mocking AEMET HTTP calls) had 142 collected tests at the time of writing, organized to keep the system’s two highest-risk areas, timezone/DST conversion and aggregation, testable in complete isolation from FastAPI, SQLAlchemy, and `httpx`:

File	Scope
<code>test_domain_time.py</code>	Timezone resolution, DST hazards, output representation
<code>test_domain_aggregation.py</code>	Bucketing, mean/max, quality filtering, DST-crossing buckets
<code>test_aemet_client.py</code>	Two-step flow, retries, 429 backoff, <code>estado: 404</code> disambiguation
<code>test_aemet_schemas.py</code>	DTO validation, "NaN" sentinel handling, domain mapping
<code>test_db_repository.py</code>	Cache hit/miss, upsert correctness, timezone round-trip
<code>test_db_models.py</code>	Schema constraints, <code>UTCDateTime</code> type behavior
<code>test_weather_service.py</code>	Chunking, cache-per-chunk, latest-available probing and TTL
<code>test_config.py</code>	Startup validation of required/optional settings
<code>test_exceptions.py</code>	Exception hierarchy shape
<code>test_logging.py</code>	Logging configuration
<code>test_main.py</code>	App factory, health check
<code>test_observations_route.py</code>	Full FastAPI app, end-to-end, AEMET mocked at the HTTP layer only

The unit/integration boundary is deliberate: unit tests (`tests/unit/`) exercise one module with everything else faked (`FakeAemetClient` and `ScriptedAemetClient` in `test_weather_service.py` implement the `AemetObservationSource` protocol with no HTTP involved at all), while integration tests (`tests/integration/test_observations_route.py`) exercise the real FastAPI app through `TestClient` against every validation and behavioral case in the specification, mocking only the AEMET HTTP layer itself via `pytest_httpx`, so the routing, dependency injection, and error-handling wiring are all genuinely exercised rather than assumed to work because the unit tests pass.

Several tests target behavior discovered only through live experimentation against the real API, encoding what was learned rather than only what was specified: the 31-day chunking boundary (`test_range_over_31_days_is_split_into_multiple_aemet_calls`), the doubling 429 cooldown (`test_429_cooldown_doubles_with_each_consecutive_retry`), and the distinction between a genuine empty result and a rejected oversized range sharing the same `estado: 404` wrapper. This is a direct consequence of the verification-first approach in Section 4: because these behaviors were confirmed empirically rather than assumed, the tests that pin them down encode actual AEMET behavior, not merely the developer’s expectations of it.

8.2 Frontend: 98 tests, behavioral rather than implementation-focused

The frontend suite (`frontend/src/**/*.test.tsx`, Vitest + React Testing Library) had 98 passing tests across 10 files at the time of writing. Every test is written from the perspective of what a user does and sees, querying by accessible role and label text (`getByRole`, `getByLabelText`) rather than by implementation detail such as CSS class names or component internals, so a purely internal refactor (renaming a prop, restructuring a component’s internal state) does not require rewriting the tests that verify its behavior.

Coverage spans form validation (required fields, start-before-end, free-text timezone escape hatch), all five request-lifecycle states (idle, loading, success, empty, error), table rendering including the null-vs-real-zero distinction that Section 6 makes load-bearing, and interaction tests confirming that changing aggregation level, station, or measurement selection actually changes the submitted query. The API layer is mocked at the `getObservations` function boundary, not at the global `fetch`, on the reasoning that the API client module (`api/client.ts`) already carries its own dedicated test suite (`client.test.ts`) covering request construction, response parsing, and error mapping, so component tests mocking `fetch` directly would duplicate that coverage rather than add to it.

Both suites are fully offline: no test in either suite depends on network access, a running counterpart service, or wall-clock timing to pass, which is what makes the 142+98 count above meaningfully reproducible rather than a snapshot of one lucky run.

9 Frontend Architecture

9.1 State as one discriminated union, not several booleans

The entire application’s request lifecycle is modeled as a single TypeScript discriminated union, `{status: 'idle' | 'loading' | 'success' | 'error', ...}` (`types/requestState.ts`), rather than a set of independent boolean flags (`isLoading`, `hasError`, `hasData`, and so on). This is not a stylistic preference: with independent booleans, the type system permits states that should never exist, such as `isLoading && hasError` both being true simultaneously, which would then require either a runtime convention nobody enforces or a comment asking future contributors not to do that. With a discriminated union, `App.tsx`’s render branches narrow `requestState`’s type based on `.status`, so accessing `requestState.data` inside the `'success'` branch is only possible because TypeScript has already proven `data` exists on that branch of the type, not because the developer remembered to check first. Every state the challenge specification calls for (initial, loading, success, empty, validation error, API error) is handled as an explicit branch in `App.tsx`, not merged into a generic “something went wrong” catch-all.

9.2 Component decomposition follows analytical weight, not file-size convenience

`App.tsx` composes a fixed sequence of sections once a query succeeds, ordered by analytical weight rather than by, say, alphabetical component name or implementation order: a KPI summary strip (`SummaryMetrics`), the time series (`ObservationsChart`), the wind-energy analysis (`WindEnergyView`), a compact query-metadata strip (`QuerySummary`), and finally the full observations table (`ObservationsTable`). This ordering encodes a judgment about what a reader of a wind-farm feasibility query would want to see first: a scannable summary before a raw table, and wind-specific analysis before generic query metadata, since wind is the measurement the business context in Section 2 actually cares about most.

`ObservationsChart` renders three vertically-stacked, single-axis panels (temperature, wind speed, pressure) synchronized via Recharts’ `syncId` rather than a single chart with two or three inde-

pendent y-axes sharing the same plot area. This replaced an earlier dual-axis design during development: overlaying $^{\circ}\text{C}$, m/s , and hPa against shared axes made the lines’ relative shapes visually comparable in a way that was an artifact of arbitrary axis scaling, not a real relationship in the underlying data, and risked a reader inferring correlation from what was actually just two independently-scaled axes happening to occupy the same pixel space. Separate panels with a synchronized crosshair preserve the ability to correlate a moment in time across all three measurements without implying a false quantitative relationship between unrelated units. Lines never connect across a `null` value, consistent with Section 6’s sparse-bucket design: a gap is a real absence of data, not an interpolation opportunity, and rendering it as a smooth connecting line would visually fabricate a trend the data does not support.

9.3 The hero and the transition to the analytical workspace

Before any query is submitted, the page shows a full-bleed hero (Hero) rather than a blank form: an entirely vector, hand-authored illustration of the Antarctic coastline with both stations marked (inline SVG, no raster imagery), a headline and call to action, and three short explainer cards (IntroFeatures) describing each measurement and the data pipeline (AEMET OpenData $\rightarrow$ validation and transformation $\rightarrow$ SQLite cache $\rightarrow$ FastAPI $\rightarrow$ the analytical workspace itself), so a first-time visitor understands what the system does and where its data comes from before being asked to fill in a form.

The screenshot shows the landing page for 'AEMET OPENDATA · ANTARCTIC STATIONS'. The hero section has a dark blue background with a white headline 'Reading the wind at the edge of the map.' and a sub-headline 'Historical temperature, pressure, and wind observations from Spain's two Antarctic research stations, queried, cached, and analyzed for wind-farm feasibility.' A 'Start a query' button is located below the sub-headline. To the right of the text is an illustration of the Antarctic coastline with two stations marked: 'Gabriel de Castilla' and 'Juan Carlos I'. Below the hero are three explainer cards: 'Antarctic stations' (2), 'Meteorological variables' (3), and 'Aggregation levels' (4). The main form includes fields for Station (Gabriel de Castilla), Start and End dates (mm/dd/yyyy), Timezone (Europe/Madrid), and Aggregation (None). A 'Query' button is at the bottom.

Figure 1: The idle-state landing page: the full-bleed polar hero with the station-marked coastline illustration, the query form, and the three measurement explainer cards, before any query has been submitted. The timezone field defaults to **Europe/Madrid**, and both date fields already show a “data available through” caption sourced from `GET /observations/latest-available` (Section 7.6).

Figure 1 shows this idle state as actually rendered: the query form sits directly below the hero, so the transition from “understanding what this tool does” to “using it” requires no navigation, only scrolling, which the hero’s own call-to-action button performs programmatically.

Once a query is submitted, the hero is unmounted, not merely hidden, and replaced by a compact header (**CompactHeader**) carrying the same visual identity in a slim strip, present across the loading, success, and error states alike. This distinction (unmount versus hide) matters for a concrete reason discovered during development: an earlier version of this transition left the page with no header at all once results were showing, because the hero component was conditionally rendered only in the idle branch and nothing replaced it, so the page briefly went headless between submitting a query and results appearing. **CompactHeader** exists specifically to close that gap, ensuring the page always carries its identity regardless of request state.

Every measurement carries one consistent color identity across the entire frontend, applied via shared CSS custom properties rather than per-component literal color values: wind speed in cyan/teal, temperature in warm orange-red, pressure in indigo/violet, used identically for chart lines, KPI card accent rails, measurement-checkbox swatches, and chart legends. Color is never the only signal carrying information; every colored element also carries an explicit text label, since color alone is not an accessible encoding for a value someone needs to act on.

Figures 2 and 3 together show the full analytical workspace a successful query produces, in the order **App.tsx** actually renders it: KPI strip, time series, wind-energy analysis, query summary, then the table.

10 Continuous Integration and Engineering Quality

Two GitHub Actions workflows (`.github/workflows/backend.yml`, `frontend.yml`) run on every push and pull request touching their respective directory, each scoped by a `paths` filter so a frontend-only change does not trigger the backend job and vice versa. The backend workflow installs the project (`pip install -e ".[dev]"`), then runs **ruff check**, **mypy** in strict mode, and **pytest** in that order, so a lint failure is reported before a slower type-check or test run even starts. The frontend workflow runs **npm ci** (a clean, lockfile-exact install, deliberately not **npm install**, so CI never silently drifts from what `package-lock.json` actually pins), then **eslint**, **vitest run**, and finally **npm run build**, which itself invokes **tsc -b** as a build prerequisite, so a type error fails the workflow at the build step without a separate, redundant **tsc -noEmit** invocation.

These workflows were added late in development, after the AEMET integration, timezone/DST handling, aggregation, caching, and frontend work described in the preceding sections were already complete, not from the project's first commit. This is stated plainly rather than left for a reviewer to reconstruct from the commit history, consistent with this report's policy throughout of describing what actually happened rather than what would read best.

What existed for most of the implementation period, and what the 142 backend and 98 frontend tests, strict-mode type checking, and linting referenced throughout this report were never contingent on, was the same checks run manually and consistently by the developer: every backend change was verified against **pytest**, **mypy** in strict mode, and **ruff** before being considered complete, and every frontend change against **vitest**, **tsc**, and **eslint** in the same way. That discipline was real, and it is the reason the test and type-check counts cited throughout this report are trustworthy even for the sections written before CI existed, but it depended on the developer remembering to apply it on every single change, with no independent mechanism verifying that a given commit actually passed everything before being pushed.

Automating that already-existing discipline into a pipeline that runs on every push and pull request, rather than only when manually invoked, is what closes the gap: a broken commit now shows as a failing check on GitHub itself, rather than being discoverable only by a reviewer who happens to clone the repository and run the suites by hand. The workflows formalize a standard that was already being met, rather than introducing one that was not.

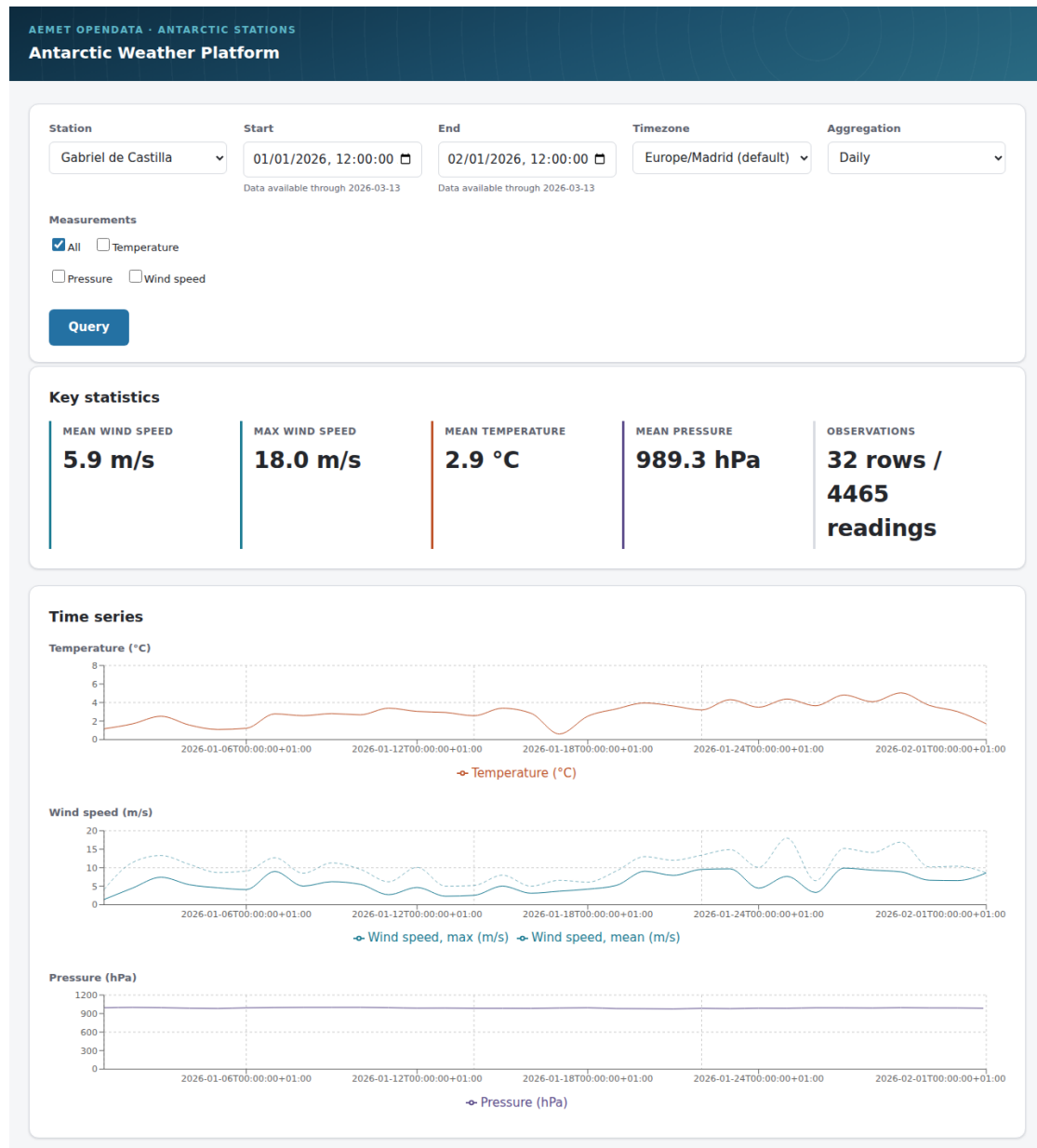


Figure 2: A successful query for Gabriel de Castilla, daily aggregation, January 2026: the compact header (replacing the hero, never leaving the page headless), the query form retaining the submitted values, the KPI strip with color-coded accent rails per measurement family, and the three synchronized time-series panels (temperature, wind speed mean/max, pressure). The dashed line in the wind panel is the bucket maximum, sharing wind’s own hue rather than a separate color, so “mean vs. max” reads as one measurement family, not two (Section 6).

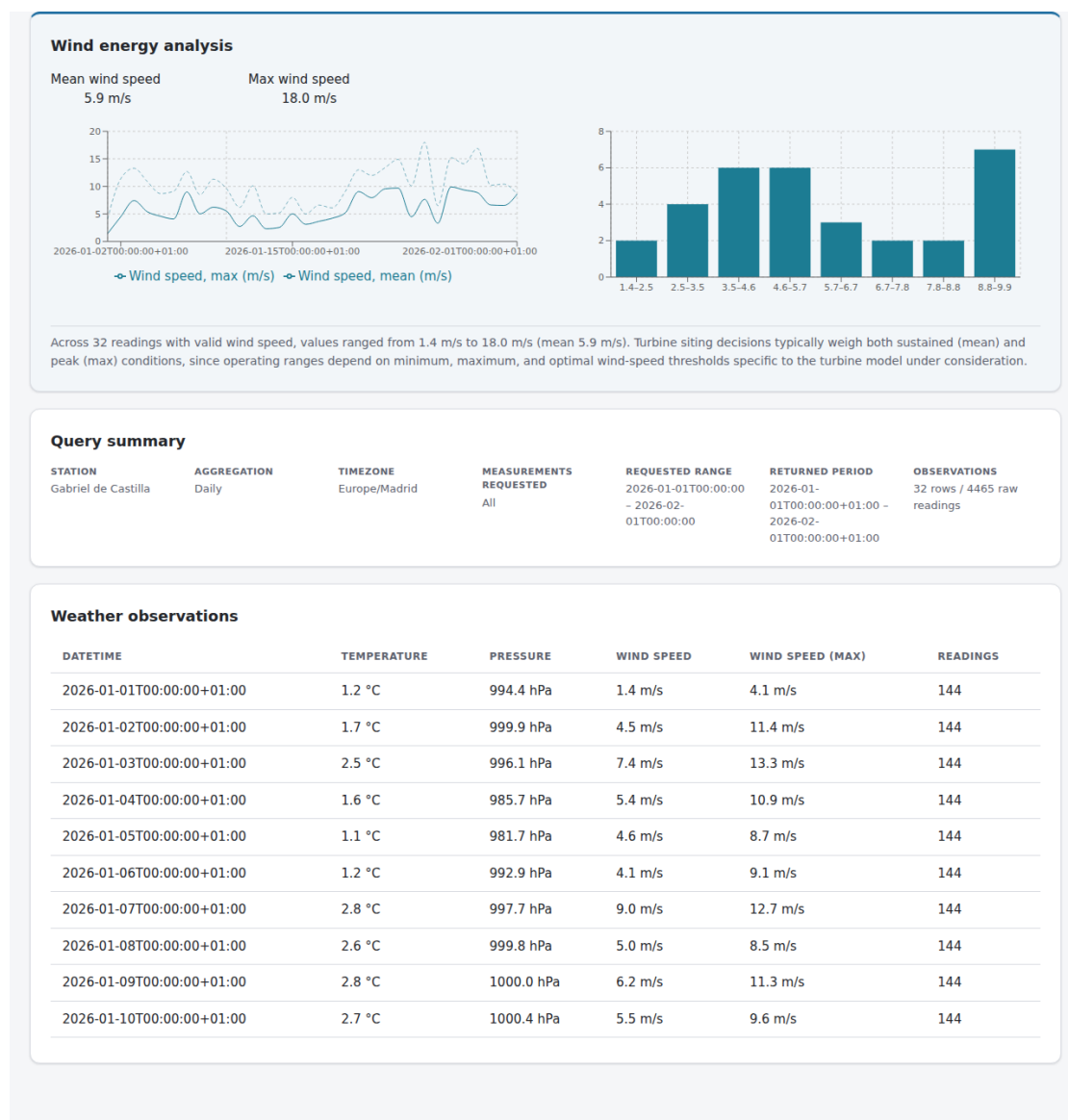


Figure 3: The same query, scrolled further down: the wind-energy analysis section (mean/max summary, a reused wind time series, and a client-side histogram of the wind-speed distribution), the query-metadata summary strip, and the observations table. No turbine-specific threshold (cut-in, rated, or cut-out speed) is printed anywhere in this view, deliberately (Section 13).

11 Security Considerations

The application’s attack surface is small and mostly bounded by what it does not do: it has no user authentication, no user-supplied data is persisted beyond the AEMET observations themselves, and it serves exactly one read-only endpoint plus a health check. The concrete measures actually in place are:

- **Secrets never committed.** The AEMET API key is read exclusively from an environment variable (`app/core/config.py`); `.env` is covered by `.gitignore`, and only `.env.example` (containing no real key) is committed.
- **Startup-time configuration validation.** A missing or malformed required setting fails immediately when the application starts, with a clear error, rather than surfacing later as a confusing failure the first time a request happens to need that setting.
- **CORS is explicitly scoped,** not left at a permissive default: `CORS_ALLOW_ORIGINS` defaults to only `http://localhost:5173` (the local Vite dev server) and the middleware permits only `GET` requests, matching the fact that this API exposes no mutating operation at all.
- **No secret ever reaches a client or an error response.** 5xx error responses return a generic message; the real exception detail, which could otherwise leak an internal table name, a config variable, or an upstream URL, is logged server-side only (Section on Error Handling, `README.md`).
- **Input validation at the boundary.** Every caller-supplied value (station, datetime format, timezone name, aggregation level, measurement selection) is validated before reaching any business logic, returning 400 with a descriptive message for a domain validation failure and 422 for a request FastAPI itself cannot parse, rather than allowing malformed input to propagate into the timezone or aggregation pipeline where a failure would be harder to trace back to its actual cause.

What this application deliberately does not need, given its actual deployment shape (a local, single-user development tool, not a multi-tenant public service): rate limiting on its own API (the AEMET client’s own outbound rate limiting, Section 4.4, is the relevant concern here, not inbound traffic to this application), a reverse proxy in front of the development server, or any credential storage, since it has no accounts to protect. These are not oversights; they are scoped out because nothing in this system’s actual deployment target creates the risk they would mitigate.

12 Scalability Considerations

The current design targets the challenge’s actual stated scale: a single-user or small-team tool querying two stations’ historical data, backed by SQLite. Several choices were made specifically because a more scalable alternative was not justified at this scale, each with a stated evolution path rather than being closed off by the current design:

- **SQLite, not PostgreSQL.** SQLite has no meaningful concurrent-write story, which is irrelevant here since this application has exactly one writer (the fetch-and-cache path) and reads are cheap. If concurrent write access ever became a real requirement, the repository’s interface (`ObservationRepository`) already isolates all persistence behind method calls with no SQL leaking into the service layer, so migrating the underlying engine to PostgreSQL would not require changing `WeatherService` or anything above it.
- **Cache-coverage checked with a single-range containment query, not an interval tree** (Section 7). The complexity cost of a general interval-merging cache is not justified by the current single-query-at-a-time usage pattern; if usage patterns changed to include many overlapping, non-adjacent range requests per station, this is the first place a more

general algorithm would be introduced.

- **Fetching on request, not a background ingestion worker.** Data is fetched from AEMET lazily, on the first request that needs an uncached range, rather than proactively ingested on a schedule. This keeps the system simple (no scheduler, no job queue) at the cost of the first request for any new range paying AEMET’s latency directly. A background worker performing scheduled ingestion is a natural evolution if this system needed to guarantee low latency on every request, including the first one for a given range.
- **No caching layer in front of SQLite.** Query volume in this application’s actual context does not approach a level where SQLite’s own read performance is a bottleneck; introducing Redis or an equivalent hot-path cache ahead of that point would be optimizing a dimension that is not currently the constraint.

Grouping and reduction inside **aggregate** are both $O(n)$ in the number of raw observations, which is the relevant complexity bound for a bounded date-range query at ~ 10 -minute granularity, at most a few thousand points even for a multi-year request once chunked, so this was not a dimension requiring further optimization at this scale.

13 Trade-offs and Limitations

This section states the deliberate trade-offs already reasoned through elsewhere in this report in one place, together with genuine limitations not otherwise framed as decisions:

- **Single-range cache containment** over a general interval algorithm (Section 7): correct and explainable for the actual usage pattern, but a request spanning two non-adjacent previously-fetched ranges is a full miss rather than a partial hit.
- **httpx over httpx2** for the production AEMET client, keeping **httpx2** only as a **TestClient** dev dependency (README, Trade-offs): a small amount of forward-risk accepted (building on a library with a long release gap) to avoid an unverified migration under real project time constraints.
- **No background ingestion, no distributed cache** (Section 12): appropriate to the current scale, with a stated evolution path if that scale changed.
- **Verification is scoped to two specific stations.** The AEMET response mapping treats every field outside the five actually used as optional, which was necessary because Gabriel de Castilla and Juan Carlos I were confirmed *not* to return an identical field set (Section 4), but this has only been verified against these two stations specifically, since they are the only two the specification requires.
- **The wind-farm feasibility framing informs presentation, not a modeling claim.** Reporting both mean and maximum wind speed, and building a dedicated wind-energy analysis view (Section 9), makes the data more useful for a feasibility assessment, but the system deliberately never states a specific turbine’s cut-in, rated, or cut-out threshold, since those are turbine-model-specific figures outside this API’s or the challenge’s actual scope; a regression test in the frontend suite asserts that no such figure ever appears in the wind-energy view.

14 Future Improvements

This section separates two different kinds of future work. Section 14.1 lists improvements that directly close a limitation already stated elsewhere in this report, ordered by the same proportionality reasoning applied throughout: the highest-value item first, each tied to a specific section rather than asserted in isolation. Section 14.2 is more speculative and product-facing, describing how the system could grow if the engagement extended beyond the challenge’s current

two-station, single-user scope; these are explicitly framed as directions, not commitments, and none of them is implied to be a small change.

14.1 Hardening: closing the gaps already stated in this report

1. **A required-status-check branch rule on main**, now that the CI workflows themselves exist (Section 10): the workflows currently report status on every push and pull request, but nothing yet blocks a merge if either one fails, so completing this means configuring GitHub’s branch protection to require both checks to pass before a pull request can merge, converting a reported signal into an enforced gate.
2. **General interval-coverage caching**, if real usage ever showed the single-range containment scope limit (Section 7) mattering in practice: a query pattern with genuinely overlapping, non-adjacent historical ranges per station would justify the added complexity of interval-tree reasoning that is not currently justified.
3. **Scheduled background ingestion**, replacing the current fetch-on-request model (Section 12), if low first-request latency ever became a hard requirement rather than an accepted trade-off.
4. **Broader station-field verification**, extending the empirical field-availability check performed for Gabriel de Castilla and Juan Carlos I (Section 4) to any additional AEMET stations, should this system’s scope ever extend beyond the two the challenge specifies.
5. **A reverse proxy and inbound rate limiting**, appropriate if this system’s deployment shape ever changed from a local, single-user tool toward a shared or public-facing service (Section 11).

14.2 Further ideas: growing beyond the challenge’s current scope

14.2.1 A dedicated mobile experience, not just a responsive layout

The current frontend is usable on a narrow viewport (the layout stacks rather than breaks), but it was designed and tested primarily against a desktop-width browser, and the three-panel synchronized time series in particular (Section 9) is not the right visualization for a phone-width screen: three stacked charts each losing most of their horizontal resolution is a worse reading experience than a single chart with a measurement switcher. A genuine mobile treatment would mean re-deriving the chart layout for small viewports rather than reflowing the existing one, likely a single active-measurement chart with a segmented control to switch between temperature, wind, and pressure, plus touch-appropriate target sizes throughout the query form (the current native `<input type="datetime-local">` controls already inherit correct touch behavior from the browser, but the measurement checkboxes and timezone dropdown were sized and spaced for a mouse cursor). This is a real, separate design pass, not a CSS media-query afterthought, which is exactly why it is scoped here as future work rather than attempted within the current timeline.

14.2.2 Database management: from a single SQLite file to an operable store

Section 12 already states why SQLite is the right choice at the current scale (one writer, cheap reads, zero operational overhead) and names PostgreSQL as the natural next engine if concurrent write access ever became a real requirement. Beyond that specific migration, a longer-lived deployment of this system would need database *management*, not just a bigger engine: a defined backup and restore procedure for `weather.db` (currently a single file with no automated backup at all, acceptable only because every row it contains is independently re-derivable from AEMET on demand), a real migration tool (Alembic, already named as the natural fit in the README’s production evolution path) once the schema has any change history worth tracking instead of the current create-if-missing startup behavior, and basic storage observability, table sizes, row

counts per station, and cache growth rate over time, so a future maintainer could answer “is the cache doing its job” from a dashboard rather than by querying SQLite directly. None of this is required at the challenge’s current scale (two stations, one deployment, one developer), which is precisely why it belongs here rather than in the hardening list above.

14.2.3 Timezone and location adaptability beyond the two Antarctic stations

The timezone pipeline (Section 5) was built to be general on purpose, accepting either an IANA name or a fixed UTC offset, resolving DST correctly, and always normalizing output to a single fixed zone, even though the current system only ever needs to reason about Europe/Madrid as that fixed output zone and two fixed-location Antarctic stations as the data source. That generality was a deliberate choice (Section 5), and it is what makes a genuine next step tractable rather than a rewrite: adding a third station, in any timezone, would not touch `domain/time.py` at all, since the module already treats “which named zone is the fixed output” as a parameter, not a hardcoded assumption, even though `OUTPUT_TIMEZONE` is currently set as a module-level constant rather than a request-time or per-deployment setting. Making that constant configurable, and pairing it with a station registry (station identifier, display name, AEMET code, and, critically, the station’s own local timezone) instead of the current two-member `Station` enum, would let this same domain logic serve any AEMET station, or in principle any similarly-shaped meteorological API, anywhere in the world, without touching the aggregation or DST correctness logic that is this system’s actual hard-won correctness guarantee. This reframes the current two-station scope not as a limitation of the architecture but as a deliberate, narrow instantiation of a pipeline that was already built general enough to outgrow it.

15 Conclusion

This system was built by treating AEMET’s actual, live behavior, not its documentation or the challenge specification’s own field table, as the final source of truth for every implementation decision that touched the external API: the two-step response indirection, the “NaN” sentinel, the undocumented quality flag, the undocumented ~31-day range limit, and the multi-retry 429 behavior were each discovered or confirmed empirically before being encoded into the client, and each is covered by a test that pins down the specific behavior discovered rather than a generic assumption about how an API like this “should” work.

The same verification-first discipline was applied internally: the two DST hazards are handled by round-tripping through UTC and checking for disagreement, not by trusting Python’s defaults; the two SQLite correctness bugs (timezone loss on round-trip, an upsert targeting the wrong key) were both caught by tests specifically designed to catch a class of subtle, silent bug, not merely to exercise the happy path; and every deliberate scope limitation, from single-range cache containment to the choice not to model wind direction, is stated as a reasoned trade-off with its own evolution path, rather than left implicit or discovered only by a reviewer reading the code.

For a period during development, this report stated a genuine gap rather than a considered trade-off: the absence of a CI pipeline (Section 10), acknowledged plainly rather than described as done, consistent with this report’s own premise that a technical report exists to be checked against running code, not to present the most flattering account available. That gap is now closed; the remaining item, gating merges on the workflows’ own status rather than only reporting it, is recorded honestly as future work (Section 14.1) rather than folded into a claim of completeness it does not yet earn.